Guide de test Loup-Garou multijoueur

Oct 22, 2026 · @Hmidouche Amine

En bref

La version multijoueur tient dans un seul fichier, `index.html` : le meneur ouvre la salle sur l'ordinateur projeté, chaque apprenant rejoint sur son téléphone avec un code de 4 lettres ou un QR code, et Firebase synchronise tout en temps réel. Compte environ 45 minutes pour tout tester la première fois.

Qui voit quoi

Écran	Ce qu'il affiche	Ce qu'on y fait
Meneur (PC + vidéoprojecteur)	Code et QR de la salle, joueurs connectés, phase, minuteur, narrateur, décompte des votes	Distribuer les rôles, lancer la nuit, passer au vote, clore le vote
Joueur (téléphone)	Sa carte secrète, ce qu'il doit faire maintenant, le minuteur	Retourner sa carte, choisir une cible la nuit (Voyante, Loups, Sorcière), voter le jour

Le parcours de test

1. Tester sans Firebase, dans plusieurs onglets de ton navigateur (mode local, 5 minutes).
2. Créer le projet Firebase et la base de données.
3. Coller la configuration dans `index.html`.
4. Refaire le test à plusieurs onglets, cette fois avec Firebase.
5. Mettre le fichier en ligne et tester avec de vrais téléphones.
6. Faire une répétition générale avec 2 ou 3 collègues ou apprenants volontaires.

Étape 1 : tester sans Firebase (5 min)

Tant que la config Firebase est vide, l'app tourne en **mode local** : tous les onglets de ton navigateur partagent la même partie. C'est le moyen le plus rapide de comprendre le déroulé avant de toucher à Firebase.

1. Télécharge `index.html` (fichier joint dans le fil) et mets-le dans un dossier `thiercelieux`.

2. Ouvre ce dossier dans VS Code et lance l'extension **Live Server** (clic droit sur `index.html` > *Open with Live Server*). Évite le double-clic sur le fichier : le lien d'invitation ne marcherait pas.
3. Dans le premier onglet, clique **Créer une salle**. Note le code de 4 lettres. Cet onglet est l'écran du meneur.
4. Ouvre 4 nouveaux onglets avec l'adresse affichée sous le code (elle finit par `?salle=ABCD`). Dans chacun, tape un prénom différent et clique **Rejoindre**.
5. Sur l'onglet meneur, clique **Distribuer les rôles**, puis dans chaque onglet joueur touche la carte. Le meneur affiche « Cartes vues : 4 / 4 ».
6. Clique **La nuit tombe** et joue la nuit en passant d'un onglet à l'autre : la Voyante choisit, les Loups choisissent, la Sorcière valide. Le meneur passe seul à l'étape suivante.
7. Le jour, clique **Passer au vote maintenant**, fais voter chaque onglet vivant. Le vote se clôt tout seul quand tout le monde a voté.
8. Continue jusqu'à l'écran de victoire, puis teste **Nouvelle partie**.

Ce que tu dois constater : chaque onglet change d'écran en moins d'une seconde quand le meneur avance, et le narrateur commente chaque mort. Un bandeau en bas rappelle que tu es en mode local.

Étape 2 : créer le projet Firebase (10 min)

Firebase est gratuit pour cet usage (offre Spark, sans carte bancaire). Il te faut juste un compte Google. Les libellés de la console changent parfois un peu, mais le chemin reste le même.

1. Va sur console.firebase.google.com et clique **Créer un projet** (ou *Ajouter un projet*).
2. Nomme-le par exemple `thiercelieux-dwwm`. Désactive Google Analytics, inutile ici. Clique **Créer le projet**.
3. Dans le menu de gauche, ouvre **Créer** (*Build*) > **Realtime Database**, puis **Créer une base de données**.
4. Emplacement : choisis **Belgique (europe-west1)**. Mode de sécurité : **Démarrer en mode test**. Valide.
5. Ouvre l'onglet **Règles** de la base, remplace tout le contenu par les règles ci-dessous, puis clique **Publier**.

```
{
  "rules": {
    "rooms": {
      "$code": {
        ".read": true,
        ".write": true
      }
    },
    ".read": false,
    ".write": false
  }
}
```

Ces règles autorisent la lecture et l'écriture uniquement sous `rooms/`, sans compte. C'est suffisant pour une salle de classe, pas pour une vraie app en production : un apprenant curieux qui ouvre les outils de développement peut lire les rôles. Tu peux d'ailleurs en faire une leçon sur la sécurité côté serveur.

6. Note l'adresse de la base affichée en haut de l'onglet **Données**. Elle ressemble à `https://thiercelieux-dwwm-default-rtdb.europe-west1.firebasedatabase.app`. Tu en auras besoin à l'étape 3.

Étape 3 : coller la config dans le fichier (5 min)

1. Dans la console Firebase, clique la roue dentée en haut à gauche > **Paramètres du projet**.
2. En bas, section **Vos applications**, clique l'icône **Web** (`</>`). Donne un surnom (`thiercelieux-web`), ne coche pas Firebase Hosting, clique **Enregistrer l'application**.
3. Firebase affiche un bloc `const firebaseConfig = { ... }`. Copie uniquement les valeurs.
4. Dans `index.html`, cherche le commentaire `1. CONFIGURATION FIREBASE` (vers la ligne 63) et remplis l'objet :

```
const firebaseConfig = {
  apiKey: "AIzaSy...",
  authDomain: "thiercelieux-dwwm.firebaseapp.com",
  databaseURL: "https://thiercelieux-dwwm-default-rtdb.europe-
west1.firebasedatabase.app",
  projectId: "thiercelieux-dwwm",
  storageBucket: "thiercelieux-dwwm.appspot.com",
  messagingSenderId: "123456789",
  appId: "1:123456789:web:abc123"
};
```

5. **Vérifie `databaseURL`.** C'est la seule ligne indispensable, et Firebase l'oublie parfois dans le bloc affiché. Si elle manque, colle l'adresse notée à l'étape 2.
6. Enregistre. Recharge l'onglet Live Server : le bandeau « Mode local » a disparu, tu es connecté à Firebase.

La clé `apiKey` n'est pas un secret : elle identifie le projet, et la sécurité vient des règles de l'étape 2. Tu peux donc publier le fichier sur GitHub.

Étape 4 : tester à plusieurs onglets avec Firebase (5 min)

Refais le test de l'étape 1, mais avec une fenêtre normale et une fenêtre de navigation privée côte à côte. Si les deux se synchronisent, la base Firebase fonctionne.

1. Onglet 1 (fenêtre normale) : **Créer une salle**.
2. Ouvre une fenêtre de navigation privée, colle le lien de la salle, rejoins avec un prénom.
3. Ouvre 2 ou 3 autres onglets joueurs (dans n'importe quelle fenêtre) et rejoins avec d'autres prénoms.
4. Dans la console Firebase, onglet **Données** de la Realtime Database : tu vois apparaître `rooms > ABCD > players` en direct. Garde cet onglet ouvert pendant le test, c'est très parlant.
5. Joue une partie complète comme à l'étape 1.

Bon à savoir : chaque onglet garde son identité de joueur tant qu'il reste ouvert. Si un joueur ferme son onglet en pleine partie, il rouvre le lien et retape exactement le même prénom pour retrouver sa carte.

Étape 5 : mettre en ligne et tester sur téléphones (10 min)

Les téléphones ont besoin d'une vraie adresse en `https`. Je te conseille GitHub Pages, que tes apprenants connaîtront de toute façon. Netlify Drop est l'alternative la plus rapide, sans compte GitHub.

Option A : GitHub Pages (recommandée)

1. Sur GitHub, crée un dépôt **public** nommé `thiercelieux`.
2. Clique **Add file > Upload files**, dépose `index.html` (avec ta config), puis **Commit changes**.
3. Va dans **Settings > Pages**. Source : **Deploy from a branch**, branche `main`, dossier `/(root)`. Clique **Save**.
4. Attends 1 à 2 minutes et recharge la page : l'adresse apparaît, du type `https://ton-pseudo.github.io/thiercelieux/`.

Option B : Netlify Drop

1. Va sur app.netlify.com/drop.
2. Glisse le dossier `thiercelieux` dans la page. Netlify te donne une adresse en `https://….netlify.app` en quelques secondes.

Le test avec téléphones

1. Ouvre l'adresse sur ton PC et clique **Créer une salle**.
2. Scanne le QR code avec 2 ou 3 téléphones (le tien, celui d'un collègue, une tablette). Chacun rejoint avec un prénom.
3. Teste au moins une fois le **Wi-Fi de l'école** et une fois la **4G** : certains réseaux scolaires bloquent les connexions temps réel.
4. Joue une partie complète. Vérifie que le minuteur défile sur les téléphones et que le vote se clôt tout seul.

Étape 6 : répétition générale et checklist

Fais une partie de 15 minutes avec 3 ou 4 personnes réelles avant la séance. C'est là que tu verras si les consignes à voix haute et le rythme de la nuit tiennent.

- [ ] Une partie complète jouée en mode local (étape 1)
- [ ] Règles Firebase publiées, `databaseURL` renseignée
- [ ] Une partie complète jouée avec Firebase à plusieurs onglets (étape 4)
- [ ] Site en ligne, adresse notée
- [ ] Test sur téléphones via le Wi-Fi de l'école et via la 4G
- [ ] Vidéoprojecteur testé : le QR code se scanne depuis le fond de la salle
- [ ] Son du PC testé si tu actives la voix du narrateur
- [ ] Durée du débat choisie (3 min conseillées pour 15 joueurs)
- [ ] Plan B prêt : la démo monoposte, jouable sans réseau

Le jour J : crée la salle 5 minutes avant, projette le QR code, et rappelle la règle d'or : téléphone face contre la table pendant la nuit.

Pour les apprenants : tu peux leur donner ce fichier comme corrigé après le défi, ou comme objectif « niveau 2 ». Il reprend la même logique d'état partagé que le contrat `village` de la démo, mais l'état vit dans Firebase au lieu d'un objet JS.

Dépannage

Ouvre la console du navigateur (F12 > Console) avant tout : la plupart des problèmes y affichent un message clair.

Symptôme	Cause probable	Solution
Le bandeau « Mode local » reste affiché	`databaseURL` vide, ou scripts Firebase bloqués	Remplis `databaseURL`, recharge avec Ctrl+F5, essaie un autre réseau
Console : `PERMISSION_DENIED`	Règles non publiées ou mal collées	Recolle les règles de l'étape 2 et clique **Publier**
Console : erreur sur l'URL de la base	`databaseURL` copiée sans `https://` ou mauvaise région	Copie l'adresse exacte affichée dans l'onglet **Données**
« Salle introuvable » sur un téléphone	Code mal tapé, ou téléphone sur une autre version du site	Scanner le QR plutôt que taper ; vérifier que tout le monde est sur la même adresse
« La partie a déjà commencé »	Joueur arrivé en retard	Il attend la partie suivante (**Nouvelle partie** garde les joueurs)
Un joueur a perdu sa carte	Onglet fermé	Rouvrir le lien et retaper exactement le même prénom
La nuit est bloquée sur une étape	Un joueur ne réagit pas (téléphone en veille)	Bouton **Passer cette étape** sur l'écran meneur
Les téléphones ne se mettent plus à jour	Wi-Fi de l'école qui coupe les connexions longues	Passer en 4G ou partage de connexion
Le QR code ne s'affiche pas	Script QR bloqué par le réseau	Donner le code de 4 lettres, à taper sur l'accueil
Le site GitHub Pages affiche une 404	Déploiement pas encore fini, ou fichier pas nommé `index.html`	Attendre 2 minutes, vérifier le nom du fichier

Nettoyer après la séance : dans l'onglet **Données**, survole `rooms` et clique la corbeille pour supprimer les anciennes salles. Pour fermer l'accès, remplace les deux `true` des règles par `false`.

Batterie de prompts en anglais

Ces prompts se collent tels quels dans Claude ou un autre assistant IA. Remplace ce qui est entre crochets, et joins `index.html` au message quand le prompt parle du code.

Tester le jeu

Plan de test complet

```
You are a QA engineer. Attached is index.html, a single-file multiplayer
Werewolf game (Firebase Realtime Database, with a local mode that syncs
browser tabs when the Firebase config is empty). The host screen is
projected; each player joins on a phone with a 4-letter room code or a QR
code.

Write a manual test plan as a table: ID, feature, steps, expected result.
Cover room creation, joining, role distribution, every night step (Seer,
Werewolves, Witch), the day vote, automatic vote closing, win screens and
"New game". Keep each test under 2 minutes.
```

Cas limites

```
Read the attached index.html and list the 15 edge cases most likely to break
a live game with 15 students. For each one, explain what happens in the
current code (quote the function name) and whether it is handled. Examples to
consider: two players with the same name, a player who joins after roles are
dealt, a phone that goes to sleep during the night, a tie vote, the host
refreshing the page, a Werewolf who is the last player alive.
```

Partie simulée pas à pas

```
Simulate a full game of the attached Werewolf app with 6 players: [Alice,
Bilal, Chloé, Driss, Emma, Farid]. Assign roles the way the code would, then
walk through each night and day, showing what the host screen and each phone
display at every step, and the exact data written under rooms/ABCD in
Firebase. Stop and flag anything in the code that would not behave as a
player expects.
```

Test de charge réaliste

```
Write a small Node.js script that uses the Firebase JS SDK to simulate [15]
players joining room [ABCD] and voting at random, so I can check that the
host screen counts votes correctly and closes the vote on its own. Use my
config below and explain how to run it.

[PASTE firebaseConfig HERE]
```

Déboguer

Erreur dans la console

```
My multiplayer Werewolf game (single index.html, Firebase Realtime Database) shows this error in the browser console:

[PASTE THE FULL ERROR AND STACK TRACE]

It happens when [describe the action, e.g. the host clicks "Night falls"]. Attached is index.html. Explain the cause in plain words, then give me the smallest fix as a before/after code snippet with the line to change.
```

Un téléphone ne se synchronise plus

```
In my Firebase Werewolf game, the host screen moves to the next phase but [one phone / all phones] stay on the previous screen. It happens on [school Wi-Fi / 4G / both], on [iPhone Safari / Android Chrome]. Attached is index.html.

List the likely causes in order of probability (listeners, onDisconnect, network blocking WebSockets, stale local state). For each, tell me how to confirm it in under 2 minutes and how to fix it.
```

La nuit reste bloquée

```
The night phase in my Werewolf game gets stuck on the [Seer / Werewolves / Witch] step even though the player has chosen. Here is the current data under rooms/[CODE] exported from the Firebase console:

[PASTE JSON]

Attached is index.html. Trace the code path that should advance the night, show me where it stops with this data, and propose a fix that also covers a player who never answers.
```

Fonctionne en local, pas avec Firebase

```
My Werewolf game works perfectly in local mode (tabs synced in one browser) but behaves differently once Firebase is configured: [describe the difference]. Attached is index.html. Compare the local-mode code path and the Firebase code path for this feature and point out every place where they do not do the same thing.
```

Firebase et sécurité

Durcir les règles

```
These are my Firebase Realtime Database rules for a classroom Werewolf game:

[PASTE RULES]

Right now any player can open DevTools and read everyone's role. Propose
stricter rules using Firebase Anonymous Authentication so that each player
can only read their own role, only the host can change the phase, and nobody
can write outside rooms/$code. Show the new rules, the code changes needed in
index.html, and what still cannot be protected without a server.
```

Comment un apprenant pourrait tricher

```
Act as a curious student who wants to cheat in this browser-based Werewolf
game (attached index.html, Firebase rules below). List every way to see
hidden roles, change votes or skip the night using only the browser DevTools.
For each trick, rate how easy it is and how to block it. I will use this as a
security lesson, so explain each one for beginner web developers.

[PASTE RULES]
```

Nettoyage automatique des salles

```
My Werewolf game creates a node under rooms/[CODE] for every game and never
deletes it. Suggest the simplest way to remove rooms older than 24 hours on
the free Spark plan (no Cloud Functions). Compare: a cleanup on room
creation, a createdAt field with rules, and a manual script. Give the code
for the option you recommend.
```

Rester dans l'offre gratuite

```
I will run a Firebase Realtime Database Werewolf game with [15] students on
phones for [2] hours, a few games per session. Estimate the simultaneous
connections, data downloaded and storage used, compare them to the current
Spark plan limits, and tell me which part of index.html would use the most
bandwidth and how to reduce it. Check the current limits on the official
Firebase pricing page rather than relying on memory.
```

Faire évoluer le jeu

Ajouter un rôle

```
Add the [Hunter / Cupid / Little Girl] role to the attached Werewolf game. Rules: [describe the role, e.g. when the Hunter dies, he immediately picks one player to kill]. Keep the existing code style, keep everything in index.html, and make it work in both local mode and Firebase mode. Give me the changes as a list of before/after snippets with where to paste each one, then a 5-step manual test for the new role.
```

Reconnexion fiable

```
In my Werewolf game, a player who closes the tab must reopen the link and type exactly the same name to get their card back. Make reconnection automatic: store a player ID in localStorage, restore the session on reload, and show the host which players are offline using Firebase presence (onDisconnect). Show the code changes and explain the edge cases (two tabs open, private browsing).
```

Accessibilité et lisibilité

```
Review the attached index.html for accessibility on phones and on a projector seen from the back of a classroom. Check contrast, font sizes, touch target sizes, colour-only information, screen reader labels and motion. Give me a prioritised list of fixes with the CSS or HTML to change, starting with what matters most for 15 students on cheap Android phones.
```

Traduire l'interface

```
Add an English/French language switch to the attached Werewolf game. Move every visible string, including the narrator lines, into one translations object, keep French as the default, and remember the choice in localStorage. Each player chooses their own language; the host screen has its own switch.
```

Découper le fichier unique

```
The attached index.html holds all the HTML, CSS and JavaScript of my Werewolf game. Propose a clean split into separate files (ES modules, no build tool) that still deploys on GitHub Pages by uploading a folder. Show the folder tree, what goes in each file, and the order to do the refactor so the game keeps working after each step.
```

Pour les apprenants

Ces prompts sont écrits pour être donnés tels quels aux apprenants. Ils demandent à l'IA d'expliquer et de guider plutôt que de faire le travail à leur place.

Comprendre le code

```
I am a beginner web developer. Attached is index.html, a multiplayer Werewolf game. Explain how it works in 10 short steps, from "the host creates a room" to "a winner is shown". For each step, name the function that does it and the Firebase path it reads or writes. Do not rewrite any code.
```

Du `village` local à Firebase

```
In our single-player demo, the whole game state lives in a JavaScript object called village. In the attached multiplayer version, the state lives in Firebase Realtime Database instead. Explain the difference with a simple analogy, then show side by side how one action (a player votes) is handled in each version. What new problems appear when the state is shared between several devices?
```

Revue de code bienveillante

```
Act as a senior developer reviewing my code for a training project (attached). Give me at most 5 comments, ordered by importance: one thing I did well, then bugs, then readability. For each comment, ask me a question that helps me find the fix myself instead of giving the solution.
```

Coach sans spoiler

```
I am building a multiplayer Werewolf game as a challenge and I am stuck on [describe the problem]. Here is my code: [PASTE CODE]. Do not give me the solution. Give me 3 hints, from the vaguest to the most precise, and wait for me to ask before showing the next one.
```

Générer des défis gradués

```
I teach a class of beginner web developers (French DWWM program). Using the attached Werewolf game, create 6 coding challenges from easy to hard (for example: change the timer duration, add a role, add reconnection, secure the rules). For each one give the goal, the files and functions involved, an estimated time, and the acceptance test the student must pass. Write the challenges in French.
```